

Frontend Coding Standards

File size rules, component discipline, the UI toolkit — and how to use AI without letting it write bad code for you



- React
- Tailwind CSS
- shadcn/ui
- Lucide Icons
- TweakCN
- Motion
- TanStack Query
- React Hook Form

1. Why These Rules Exist	
2. File & Component Size Rules	
3. When and How to Split a Component	
4. Styling Rules with Tailwind CSS	
5. The Complete Frontend Toolkit	
6. Using AI the Right Way	
7. Naming & Code Style	
8. Golden Rules Checklist	

1. Why These Rules Exist

Frontend code rots differently from backend code. A backend file grows because logic piles up. A frontend file grows because "just one more div" keeps feeling harmless — until one component file is 900 lines long, does five unrelated things, and nobody (including the person who wrote it) wants to touch it.

THINK OF COMPONENTS LIKE LEGO BRICKS

A single Lego brick is simple and reusable — you can build anything with enough of them. A single Lego piece pre-molded into an entire castle is impressive once, and useless for anything else. Every component you write should be closer to a brick than a castle: small, single-purpose, reusable in more than one place.

2. File & Component Size Rules

THE 300-LINE RULE

If a component file crosses roughly **250-300 lines**, that's not a target to hit — it's a warning sign. It means the component is very likely doing more than one job and needs to be split. Treat it as a hard ceiling, not a guideline to stretch.

What Usually Causes a File to Grow Too Big

- A page component that also contains all the data-fetching logic inline
- Repeated JSX blocks copy-pasted 3 times instead of extracted into a sub-component
- Multiple unrelated UI sections (header + form + list + modal) all living in one file
- Business logic (calculations, formatting, validation) written directly inside the component instead of a hook or util function

Rule of One

Every component should be able to answer "what is your one job?" in a single sentence. If you need "and" to describe it, it's probably two components.

TOO MUCH RESPONSIBILITY

"This component fetches the term list, renders the search bar, shows each term card, **and** handles the share popup."

ONE JOB EACH

SearchBar , TermList , TermCard , ShareModal — four small components, one page that assembles them.

3. When and How to Split a Component

Signal 1 — Repeated JSX

If you're writing similar-looking JSX more than once, extract it.

```
// Before — repeated 3 times inside TermListPage.jsx
<div className="border rounded-lg p-4">
  <h3>{term.word}</h3>
  <p>{term.meaning}</p>
</div>

// After — extracted into components/TermCard.jsx
<TermCard word={term.word} meaning={term.meaning} />
```

Signal 2 — Logic That Isn't About Rendering

Data fetching, debouncing a search input, or managing complex state — pull it into a custom hook.

```
// hooks/useTermSearch.js
function useTermSearch(query) {
  const [results, setResults] = useState([]);
  useEffect(() => {
    // fetch logic here
  }, [query]);
  return results;
}

// component just consumes it — stays small and readable
const results = useTermSearch(query);
```

Signal 3 — A Page Doing Too Much

A page component's job is to **assemble** smaller components, not contain all the logic and markup itself.

Good mental model: pages/ = layout and assembly. components/ = the actual reusable building blocks. hooks/ and services/ = logic that isn't visual at all. If you're scrolling for more than a few seconds inside one file to find something, it's time to split.

4. Styling Rules with Tailwind CSS

- Use Tailwind utility classes as the default — avoid custom CSS files unless something genuinely can't be done with utilities
- Avoid inline `style={{ }}` objects — they bypass your design system and make styles inconsistent across the app
- Use your Tailwind config's design tokens (spacing, colors) instead of arbitrary values like `mt-[13px]` — arbitrary values are a sign you're guessing instead of following a system
- Group related classes together when writing them: layout → spacing → typography → color, so class lists stay scannable

```
// Avoid
<div style={{ marginTop: "13px", color: "#333333" }}>

// Prefer
<div className="mt-3 text-gray-800">
```

5. The Complete Frontend Toolkit

Don't build every button, form, animation, or notification from scratch — that's slow, inconsistent, and usually where the ugliest AI-generated UI comes from. This is the full toolkit, grouped by what problem each tool solves. You won't need all of it for every feature, but you should know what exists before reaching for a from-scratch solution.

5.1 Component Foundations

shadcn/ui
Pre-built, accessible components — buttons, dialogs, cards, dropdowns, forms. The code is copied directly into your project, so you can open and edit it like any other component. Your default choice before building any common UI element from scratch.

Radix UI
The accessible, unstyled primitives that shadcn/ui is built on top of. You usually won't touch this directly, but if shadcn doesn't have the exact component you need, Radix probably has the raw building block for it.

5.2 Icons

Lucide Icons
A consistent, clean icon set (`lucide-react`). Use it for every icon in the app instead of mixing emoji, random PNGs, and inconsistent icon styles.

Heroicons
A solid alternative icon set if you need a style Lucide doesn't cover. Pick one icon library per project and stick to it — never mix two icon sets in the same app.

5.3 Theming & Design Tokens

TweakCN
A visual theme editor for shadcn/ui. Instead of guessing colors and spacing by trial and error, generate a consistent, professional-looking theme (colors, radius, fonts) and drop it straight into your Tailwind config.

Tailwind CSS
Utility-first CSS framework. Powers both your custom styling and everything shadcn/ui generates — one consistent styling language across the whole app.

5.4 Animation

Motion (Framer Motion)
The standard library for React animation — page transitions, hover effects, entrance animations. Use it instead of hand-writing CSS keyframes for anything interactive.

tailwindcss-animate
A Tailwind plugin for simple animations (fade, slide, spin) using just utility classes. Good for small transitions where a full animation library is overkill — shadcn/ui already uses this internally.

5.5 Forms & Validation

React Hook Form
Handles form state, submission, and error display without re-rendering the whole form on every keystroke. Use it for any form beyond a single input — search bars can stay simple, but signup/login/multi-field forms should use this.

Zod
Defines validation rules as code (e.g. "email must be valid," "password min 8 characters") and pairs directly with React Hook Form. Keeps validation logic out of your components and in one readable place.

5.6 State & Data Fetching

TanStack Query
Manages server data — fetching, caching, loading/error states, refetching. Instead of writing `useEffect` + `useState` for every API call, this handles it consistently and prevents duplicate requests.

Zustand
Lightweight global state management — for things like "is the user logged in" or "is dark mode on" that many components need. Simpler than Redux, use it only when React Context starts feeling clunky.

5.7 Utility Helpers

clsx + tailwind-merge

class-variance-authority (cva)

Combine conditional class names cleanly (e.g. "this class only if active") without messy template strings, and safely merge Tailwind classes without conflicts. Usually wrapped in one `cn()` helper function — shadcn/ui ships with this already set up.

Defines component style "variants" cleanly — e.g. a Button with `variant="outline"` vs `variant="destructive"` . This is how shadcn/ui components handle different visual styles without messy if/else chains.

5.8 Feedback & Notifications

Sonner
Clean, simple toast notifications ("Term saved to favorites", "Something went wrong"). Use it instead of browser `alert()` popups, which look unprofessional and block the whole page.

shadcn Skeleton
Loading placeholders that mimic your content's shape while data is fetching. Use it instead of a blank screen or a plain spinner — it makes the app feel faster even when load time is the same.

5.9 Data Visualization

Recharts
If your app needs any charts or graphs (e.g. an admin dashboard showing most-shared terms), use this instead of building SVG charts by hand.

date-fns
Formatting and manipulating dates ("2 days ago", "Aug 14, 2026") without writing manual date math, which is a common source of subtle bugs.

5.10 Fonts & Typography

- Use Google Fonts (or similar) rather than the browser default — pick one heading font and one body font, and stop there
- Limit yourself to 2-3 font sizes for body text and 2-3 for headings — a huge range of font sizes is a common sign of an inconsistent, unplanned design
- Set your font choices once in the Tailwind config, not inline in components, so the whole app stays consistent automatically

5.11 Code Quality Tools

- **ESLint** — catches likely bugs and enforces consistent code style automatically, before a teammate has to point it out in review
- **Prettier** — auto-formats code (indentation, quotes, spacing) so the whole team's code looks the same regardless of who wrote it
- Run both before every commit — most editors can be set to format-on-save so this is automatic, not a manual step you forget

Why this matters: a component built with shadcn + Tailwind + Lucide + proper form/state handling looks and behaves professionally by default, because accessibility, loading states, validation, and consistent styling are already handled correctly. A component built from scratch in a rush usually gets one of these wrong — and that gap is exactly what makes rushed or blindly-AI-generated UI look "off."

```
// Example: shadcn Button + Lucide icon + cn() utility together
import { Button } from "@/components/ui/button";
import { Share2 } from "lucide-react";
import { cn } from "@/lib/utils";

<Button
  variant="outline"
  className={cn("gap-2", isSaving && "opacity-50")}
>
  <Share2 className="h-4 w-4" /> Share
</Button>
```

5.12 Quick Reference — What to Reach For

You Need To...	Reach For
Build a button, card, modal, dropdown	shadcn/ui
Add an icon	Lucide Icons
Set a color/theme for the app	TweakCN + Tailwind config
Animate something appearing/moving	Motion or tailwindcss-animate
Build a form with validation	React Hook Form + Zod
Fetch and cache data from the API	TanStack Query
Share state across many components	Zustand or React Context
Conditionally combine class names	clsx / tailwind-merge (cn)
Show a success/error message	Sonner (toast)
Show a chart or graph	Recharts
Format a date	date-fns
Keep code style consistent	ESLint + Prettier

6. Using AI the Right Way

AI tools are allowed and genuinely useful — but treat AI output the way you'd treat code from a junior engineer: fast, often helpful, but not something you merge without reading and understanding it first.

THE RULE THAT MATTERS MOST

If you can't explain what a piece of AI-generated code does — line by line, to a teammate — you don't understand it well enough to have accepted it. Blind copy-paste is not allowed, and it will be visible in review.

Common Bad Patterns AI Tends to Produce

- **One giant component** — AI defaults to writing everything in a single file unless told otherwise, straight into 300+ line territory
- **Inline styles instead of Tailwind** — AI often reaches for `style={{ }}` objects rather than your project's utility classes
- **Reinvented UI instead of shadcn** — AI will happily hand-build a custom modal or button from raw divs instead of using the shadcn component already in your project
- **Hardcoded data** — sample text and values baked directly into the component instead of passed in as props
- **Missing accessibility basics** — no alt text, no aria-labels, non-semantic tags used where a button or link should be
- **Generic, meaningless names** — `handleClick` , `data` , `temp` instead of names that describe what's actually happening

Before You Accept Any AI-Generated Code, Check:

- Have I read every line, not just skimmed the result?
- Does it stay under the file-size limit — or does it need splitting before I even commit it?
- Did it use shadcn/Tailwind/Lucide where those already fit, instead of reinventing them?
- Are the names it used actually meaningful, or do I need to rename things?
- Could I rebuild this from memory if asked to explain it tomorrow?

KEEP AN AI USAGE LOG

For every meaningful use of AI, note: where you used it, why, and what you accepted or rejected from its output. This isn't busywork — it's the same discipline expected in real engineering teams, and it's what turns "AI wrote my code" into "I used AI as a tool and made the calls myself."

7. Naming & Code Style

- Component files: `PascalCase` — `TermCard.jsx` , not `termcard.jsx`
- One component per file, and the file name matches the component name exactly
- Props should be named for what they represent, not their type — `meaning` , not `text1`
- Boolean props/state start with `is` / `has` / `should` — `isLoading` , not `loading1`
- Event handlers start with `handle` — `handleShareClick` , not `onClick2`
- Keep components pure where possible — same props in, same UI out, no hidden side effects buried in render

8. Golden Rules Checklist

BEFORE YOU COMMIT ANY FRONTEND CODE, CONFIRM:

- Is this file under ~300 lines? If not, what should be split out?
- Does this component do exactly one job?
- Did I use shadcn/ui, Lucide, and Tailwind instead of building UI elements from scratch?
- Are there any inline styles I should convert to Tailwind classes?
- If AI helped write this — have I read, understood, and could I explain every line?
- Are my component, prop, and function names actually descriptive?